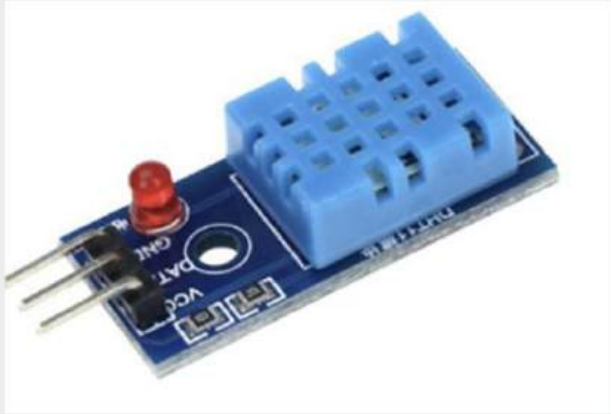
Sensor Project

SR04

DHT11

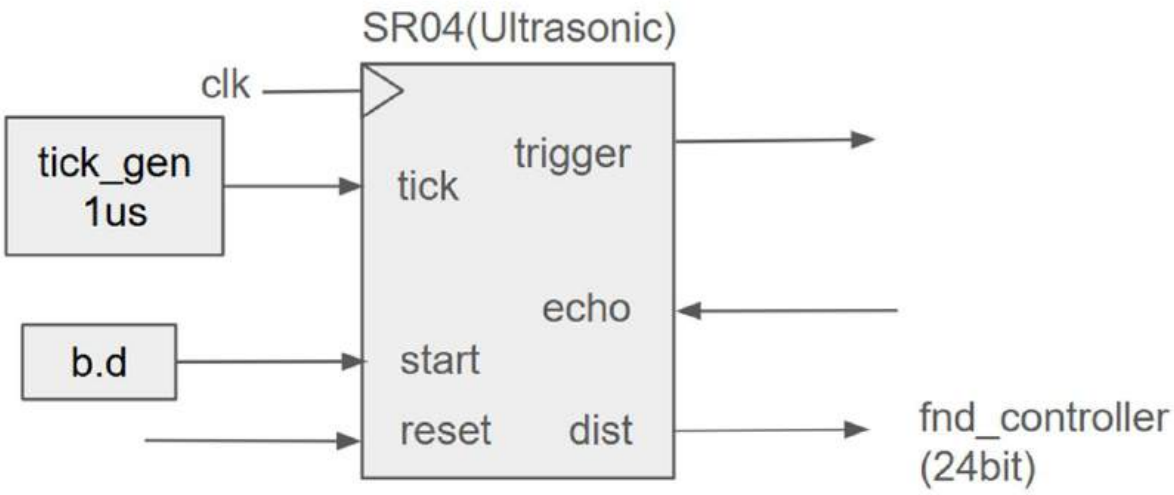


Verilog로 두 센서를 활용하는 디지털 시스템 설계

Module Design

SR04 - Ultrasonic Sensor

INTRODUCTION



Distance_Sensor_Module

(Block Diagram)

INTRODUCTION

1cm → 58us(왕복시간)
400cm → 23.2ms(왕복시간)
(TIMEOUT)

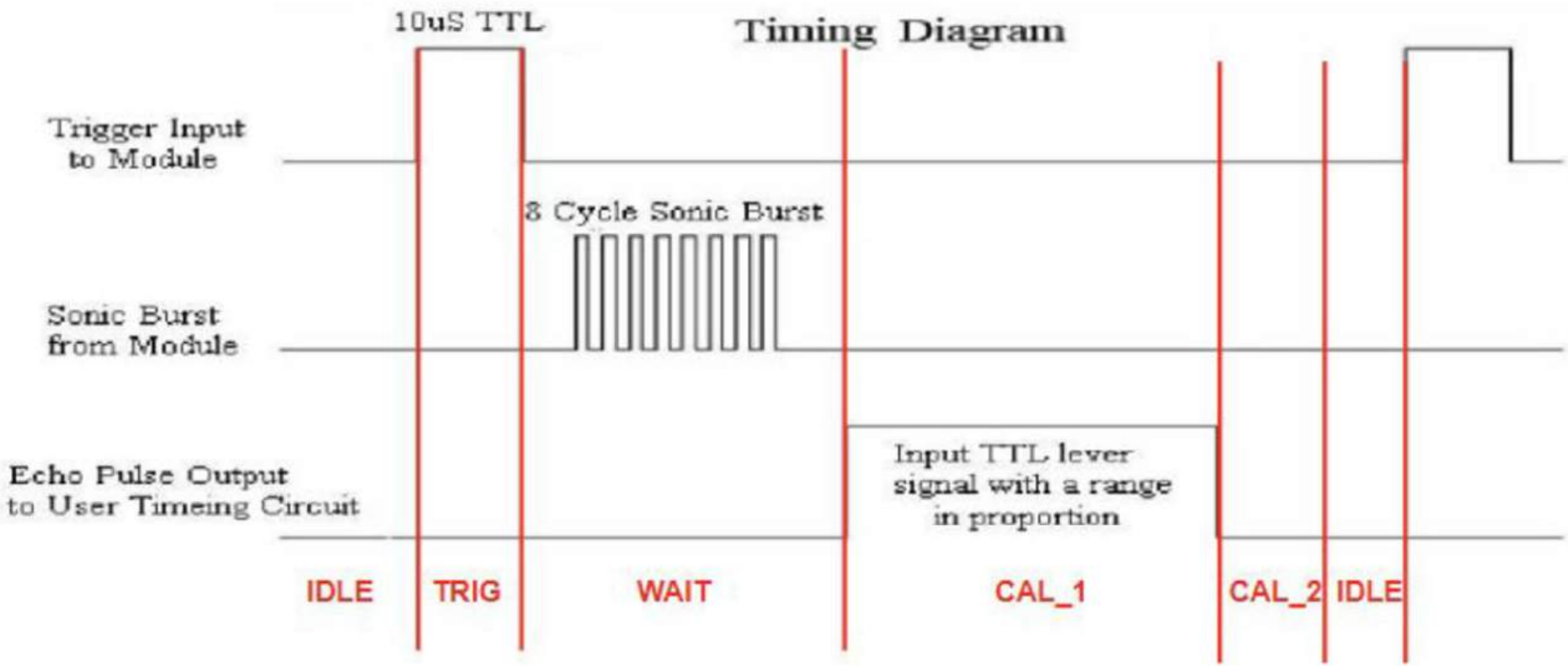
Electric Parameter

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm
MeasuringAngle	15 degree
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

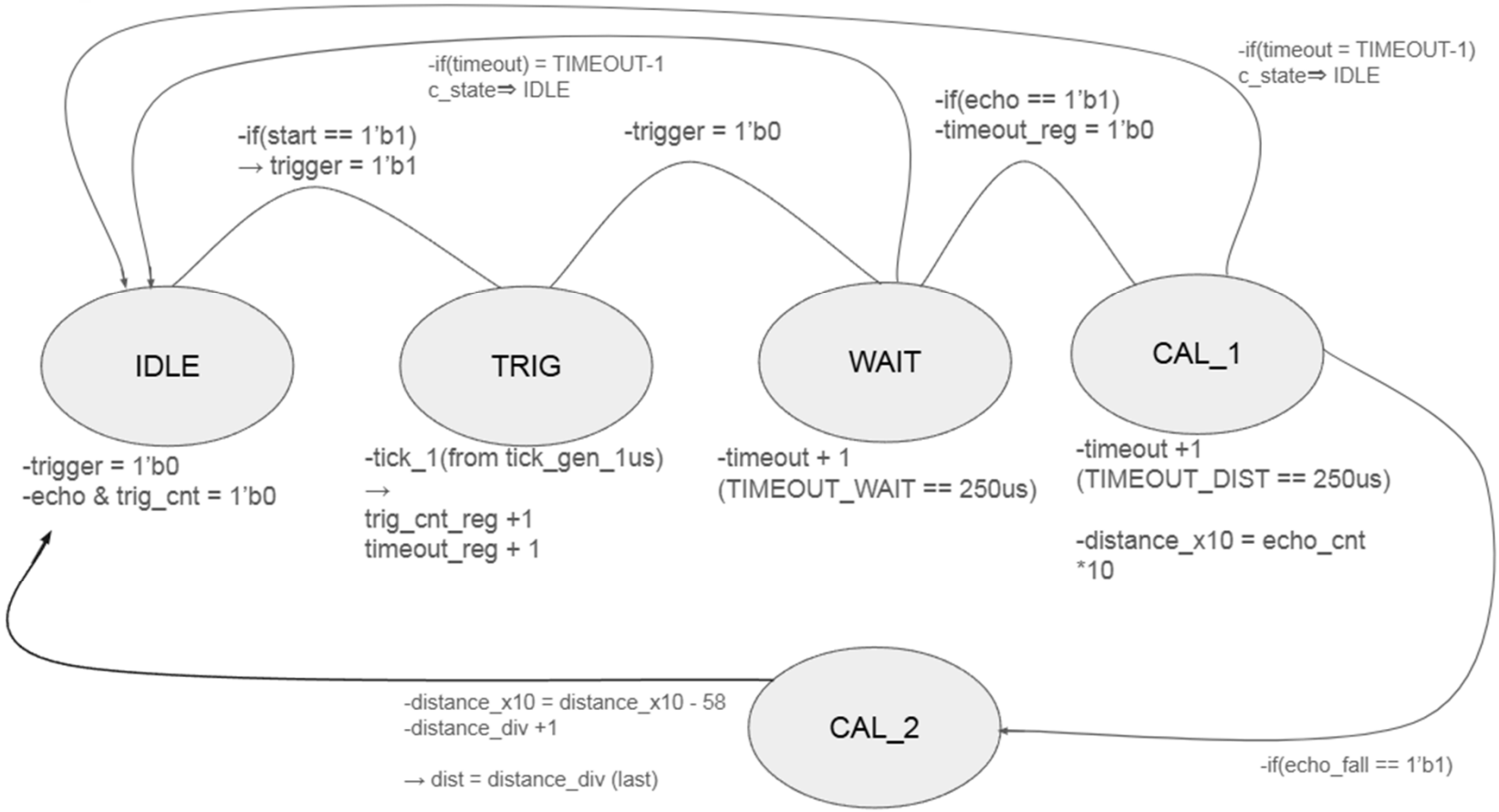
```
TRIG: begin
    trigger <= 1'b1;
    if (tick_1) begin
        if (trig_cnt == 4'd10) begin
            trigger        <= 1'b0;
            trig_cnt       <= 4'd0;
            timeout_cnt <= 15'd0;
            c_state        <= WAIT;
        end else begin
            trig_cnt <= trig_cnt + 1;
        end
    end
end
```

```
CALC_1: begin
    if (echo_fall) begin
        distance_x10 <= echo_cnt * 10; // for fixed-point scaling XXX.X cm
        distance_div <= 19'd0;
        c_state <= CALC_2;
    end else if (tick_1) begin
        echo_cnt <= echo_cnt + 1;
        if (timeout_cnt >= TIMEOUT_CALC - 1) begin
            dist <= 13'd0;
            c_state  <= IDLE;
        end else begin
            timeout_cnt <= timeout_cnt + 1;
        end
    end
end
```

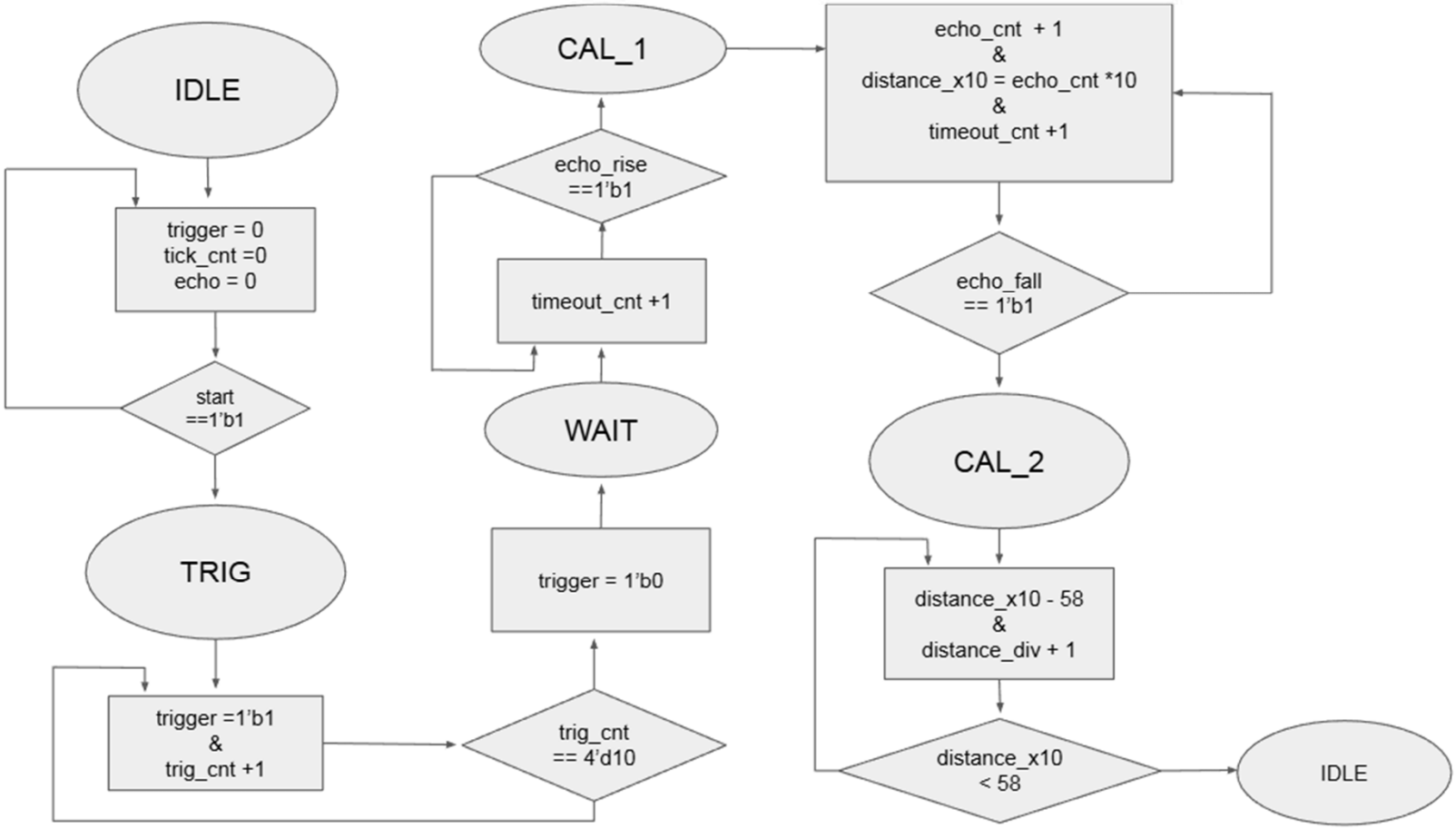
STATE, FSM/ASM



STATE, FSM/ASM

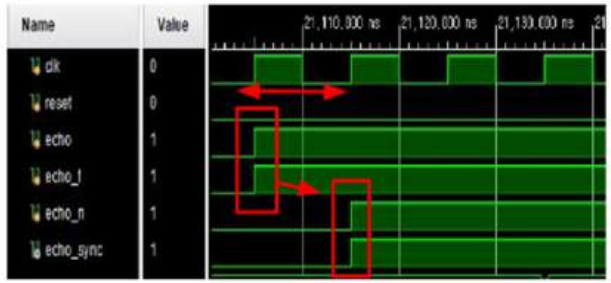


STATE, FSM/ASM



SYNCHRONIZER & EDGE DETECTION

```
assign echo_sync = echo_n;

// Synchronize echo
always @(posedge clk or posedge reset) begin
    if (reset) begin
        echo_f <= 1'b0;
        echo_n <= 1'b0;
    end else begin
        echo_f <= echo;
        echo_n <= echo_f;
    end
end
```



(Almost same clk)

- Syncronizer → 외부 신호로 인한 metastablity 방지
- 1 CLK 뒤 echo_n 전달

SYNCHRONIZER & EDGE DETECTION

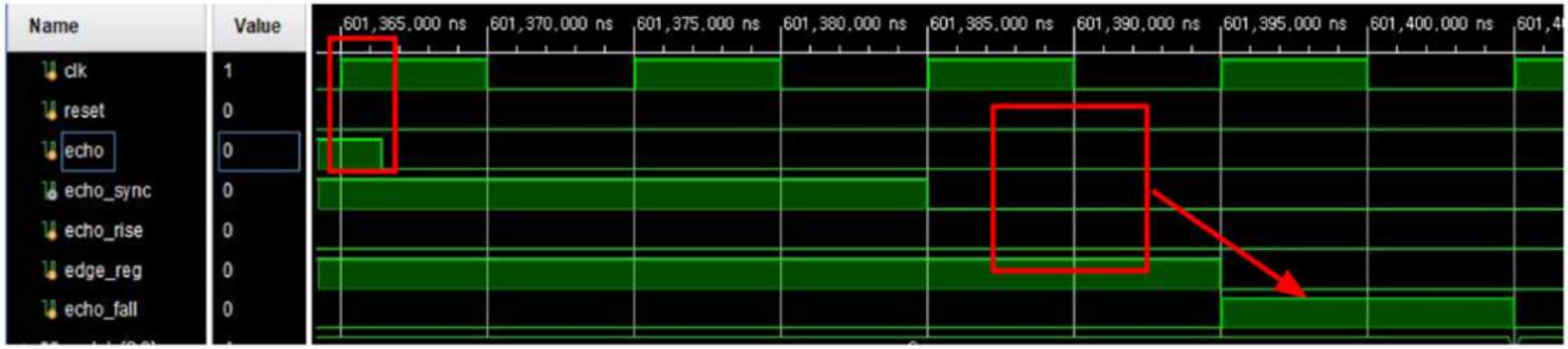
```
43    #10_000;
44    #130.7;
45    echo =1;
46    #580_000;
```

#130.7 → CLK 비동기

```
72    // edge detection
73    always @(posedge clk or posedge reset) begin
74        if (reset) begin
75            edge_reg  <= 1'b0;
76            echo_rise <= 1'b0;
77            echo_fall <= 1'b0;
78        end else begin
79            echo_rise <= (~edge_reg) & echo_sync;
80            echo_fall <= edge_reg & (~echo_sync);
81            edge_reg  <= echo_sync;
82        end
83    end
```

- echo의 신호를 인위적으로 클록과 강제 불일치
- Detection 정상동작 (echo_rise)

SYNCHRONIZER & EDGE DETECTION



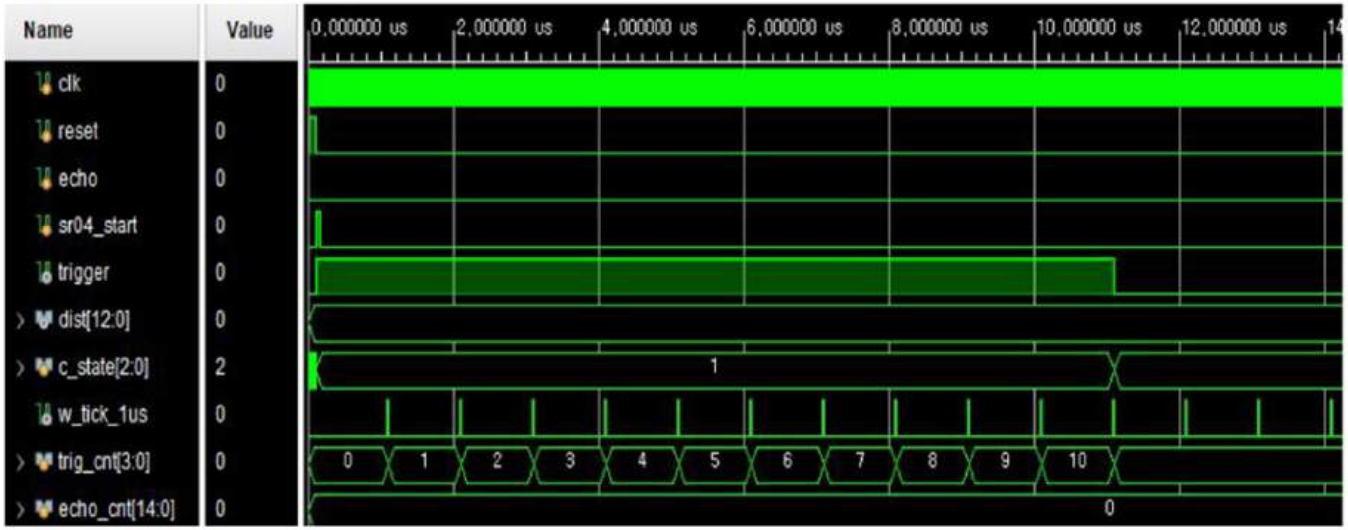

```
44      #130.7;
45      echo =1;
46      #580_000;
47      #130.7;
48      echo =0;
```

#130.7 → CLK 비동기

```
72      // edge detection
73      always @(posedge clk or posedge reset) begin
74          if (reset) begin
75              edge_reg  <= 1'b0;
76              echo_rise <= 1'b0;
77              echo_fall <= 1'b0;
78          end else begin
79              echo_rise <= (~edge_reg) & echo_sync;
80              echo_fall <= edge_reg & (~echo_sync);
81              edge_reg  <= echo_sync;
82          end
83      end
```

- echo의 신호를 인위적으로 클록과 강제 불일치
- Detection 정상동작 (echo_fall)

FSM CODE & SIMULATION

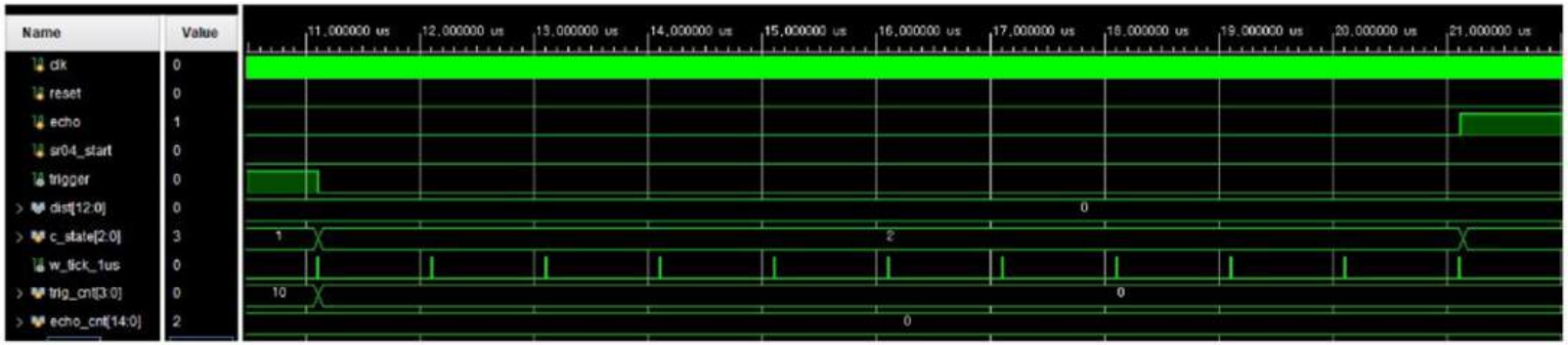



```
113            TRIG: begin
114                trigger <= 1'b1;
115                if (tick_1) begin
116                    if (trig_cnt == 4'd10) begin  //until 10us
117                        trigger        <= 1'b0;
118                        trig_cnt       <= 4'd0;
119                        timeout_cnt <= 15'd0;
120                        c_state        <= WAIT;
121                    end else begin
122                        trig_cnt <= trig_cnt + 1;
123                    end
124                end
125            end
```

- trigger가 상승 → c_state는 1(START)로 이동
- trig_cnt가 작동 → 1(START)의 trigger의 tick을 count

FSM CODE & SIMULATION

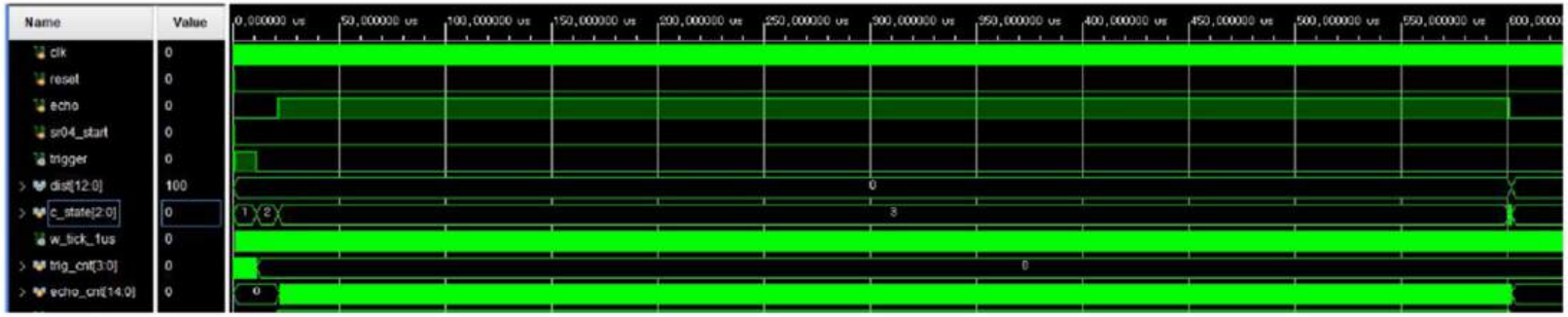
```
WAIT: begin
    if (echo_rise) begin
        echo_cnt    <= 15'd0;
        timeout_cnt <= 15'd0;
        c_state     <= CALC_1;
    end else begin
        if (tick_1) begin
            if (timeout_cnt >= TIMEOUT_WAIT - 1) begin
                dist <= 13'd0;
                c_state  <= IDLE;
            end else begin
                timeout_cnt <= timeout_cnt + 1;
            end
        end
    end
end
```



- c_state의 변환 (1→ 2(WAIT)) → 조건(echo가 들어오기 까지)
- 어떤 tick_cnt도 동작하지 않음

FSM CODE & SIMULATION

```
CALC_1: begin
    if (echo_fall) begin
        distance_x10 <= echo_cnt * 10;
        distance_div <= 19'd0;
        c_state <= CALC_2;
    end else if (tick_1) begin
        echo_cnt <= echo_cnt + 1;
        if (timeout_cnt >= TIMEOUT_CALC_1 - 1) begin
            dist <= 13'd0;
            c_state  <= IDLE;
        end else begin
            timeout_cnt <= timeout_cnt + 1;
        end
    end
end
```



- echo의 신호가 상승 c_state (2→3(CAL_1))변경→ echo_tick_cnt 동작
- distance_x10 = echo_cnt * 10 (for dislay dec)

FSM CODE & SIMULATION

```
            //original is 'distance /58'
            // because slack
            CALC_2: begin
                if (distance_x10 >= 19'd58) begin
                    distance_x10 <= distance_x10 - 19'd58;
                    distance_div <= distance_div + 1'b1;
                end else begin
                    dist <= distance_div;
                    c_state  <= IDLE;
                end
            end
            default: c_state <= IDLE;
        endcase
    end
end
```

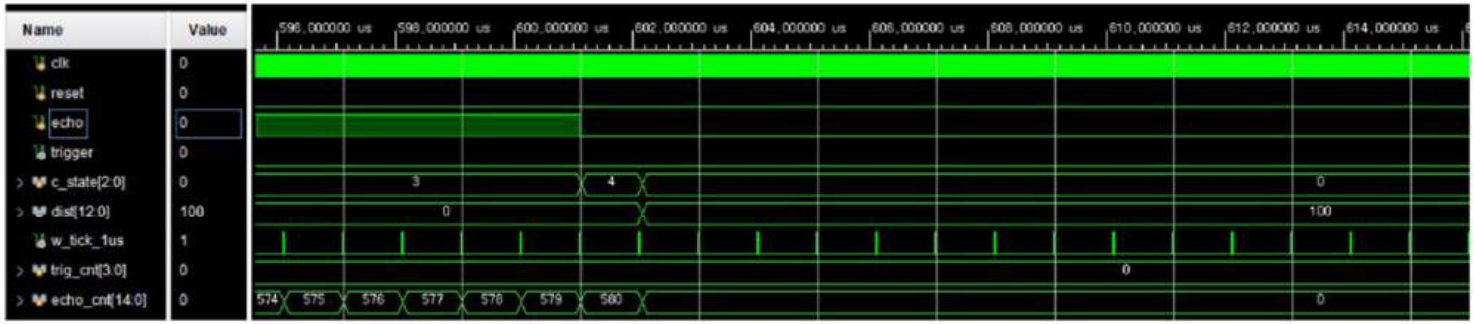
Timing diagram

The Timing diagram is shown below. You only need to supply a short 10uS pulse to the trigger input to start the ranging, and then the module will send out an 8 cycle burst of ultrasound at 40 kHz and raise its echo. The Echo is a distance object that is pulse width and the range in proportion .You can calculate the range through the time interval between sending trigger signal and receiving echo signal. Formula: uS / 58 = centimeters or uS / 148 =inch; or: the range = high level time * velocity (340M/S) / 2; we suggest to use over 60ms measurement cycle, in order to prevent trigger signal to the echo signal.

- 나누기 연산 → if(조건)을 통한 감산방식 사용(slack 방지)

FSM CODE & SIMULATION

```
            //original is 'distance /58'
            // because slack
            CALC_2: begin
                if (distance_x10 >= 19'd58) begin
                    distance_x10 <= distance_x10 - 19'd58;
                    distance_div <= distance_div + 1'b1;
                end else begin
                    dist <= distance_div;
                    c_state  <= IDLE;
                end
            end
            default: c_state <= IDLE;
        endcase
    end
end
```



- echo 하강엣지(edge_fall) 기준 → 580의 echo_cnt 형성
- distance_x10 조건방식으로 반복 → dist (100{for display dec})

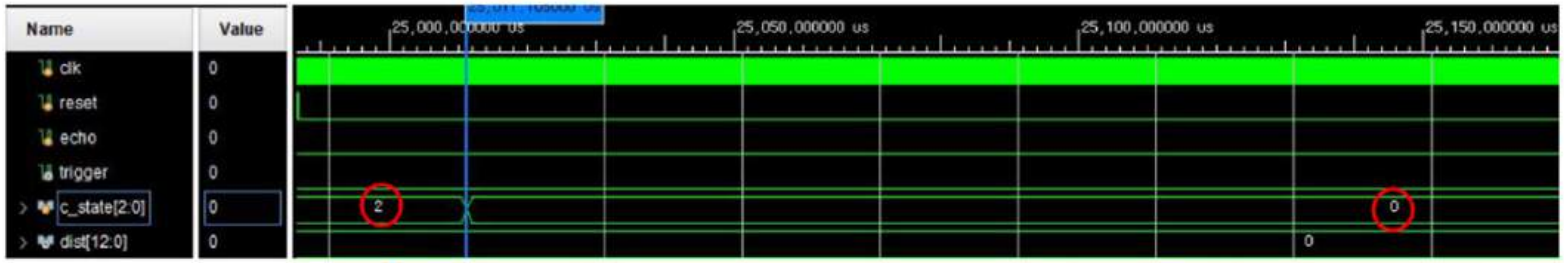
TIMEOUT FUNCTION

```
module sr04_ctrl (
    input            clk,
    input            reset,
    input            tick_1,
    input            start,
    input            echo,
    output reg       trigger,
    output reg [12:0] dist
);

    localparam IDLE = 3'd0, TRIG = 3'd1, WAIT = 3'd2, CALC_1 = 3'd3, CALC_2 = 3'd4;

    parameter TIMEOUT_WAIT = 25000; // 25ms
    parameter TIMEOUT_CALC_1 = 25000; // 25ms

    reg [ 2:0] c_state;
    reg [ 3:0] trig_cnt;
    reg [14:0] echo_cnt;
    reg [14:0] timeout_cnt;
    reg [18:0] distance_x10;
    reg [18:0] distance_div;
```

Case 1. c_state 2(WAIT) → No echo signal == 28ms (TIMEOUT_WAIT == 25ms)

Case 2. c_state 3(CALC_1) → echo == 28ms (TIMEOUT_CAL_1 == 25ms)

TIMEOUT FUNCTION




```
if (tick_1) begin
    if (timeout_cnt >= TIMEOUT_WAIT - 1) begin
        dist <= 13'd0;
        c_state  <= IDLE;
    end else begin
        timeout_cnt <= timeout_cnt + 1;
    end
end
end
end
```

TESTBENCH

```
sr04_start =1;
#50;
sr04_start =0;
wait(trigger == 1);
wait(trigger == 0);
#28_000_000;
echo =1;
#580_000;
echo =0;

#10_000;
```

- WAIT(2) → No echo Signal == 28ms 동작
- timeout_cnt >= 25ms → IDLE(0)

TIMEOUT FUNCTION

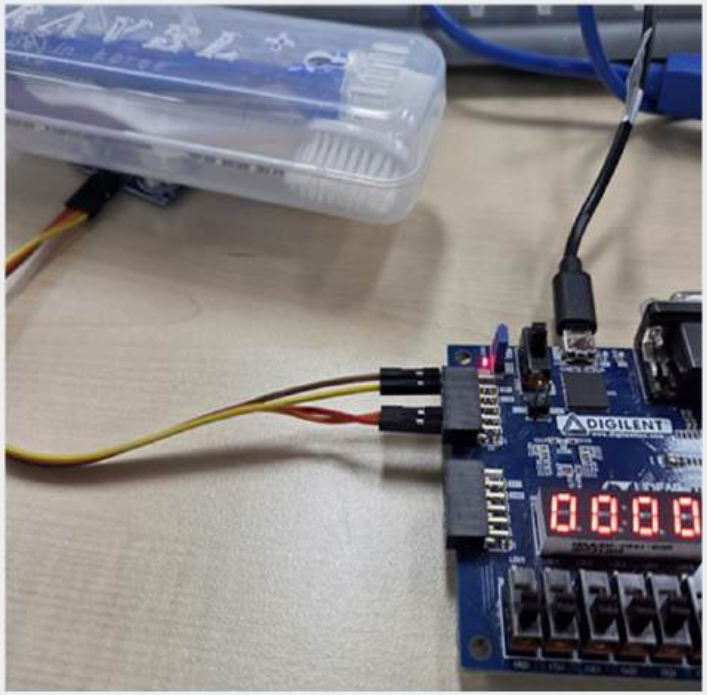
```
CALC_1: begin
    if (echo_fall) begin
        distance_x10 <= echo_cnt * 10; // for fixed-point scaling XXX.X cm
        distance_div <= 19'd0;
        c_state <= CALC_2;
    end else if (tick_1) begin
        echo_cnt <= echo_cnt + 1;
        if (timeout_cnt >= TIMEOUT_CALC_1 - 1) begin
            dist <= 13'd0;
            c_state <= IDLE;
        end else begin
            timeout_cnt <= timeout_cnt + 1;
        end
    end
end
```

TESTBENCH

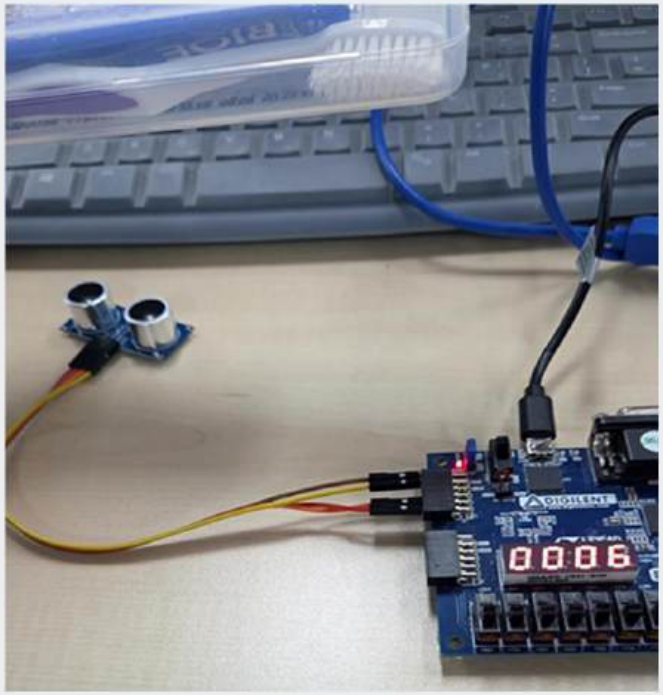
```
sr04_start =1;
#50;
sr04_start =0;
wait(trigger == 1);
wait(trigger == 0);
#10_000;
echo =1;
#28_000_000;
echo =0;

#10_000;
```

- CAL_1(3)에서 echo == 28ms 동작
- timeout_cnt >= 25ms → IDLE(0)

SR04 시연



거리 0

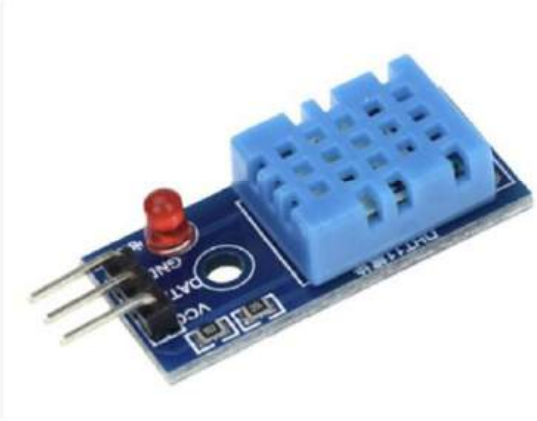


거리가 생기자 값 증가

Module Design

DHT 11 - Sensor

DHT 11 SENSOR



DHT 11

[온도/습도 센서]

기능 : 온도 및 습도 측정

통신 방식 : Single Wire - Half Duplex

전송 데이터 : 총 40bit (MSB)

측정 범위 : 20 ~90%RH / 0 ~ 50°C

정확도 : ±5%RH / ±2°C

응답 시간 : 10s(typical) / 6s(min) ~ 30s(max)

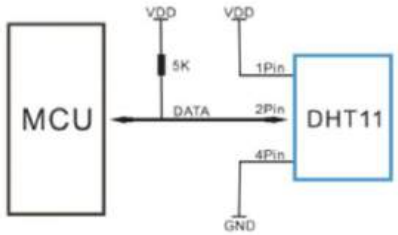
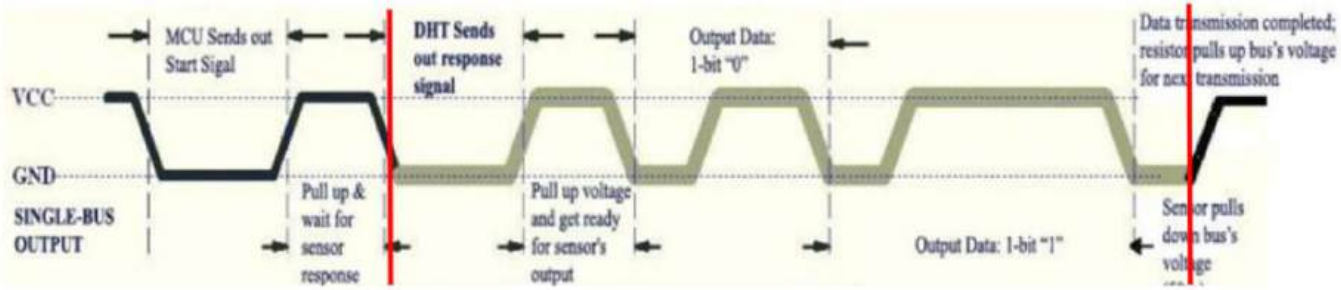
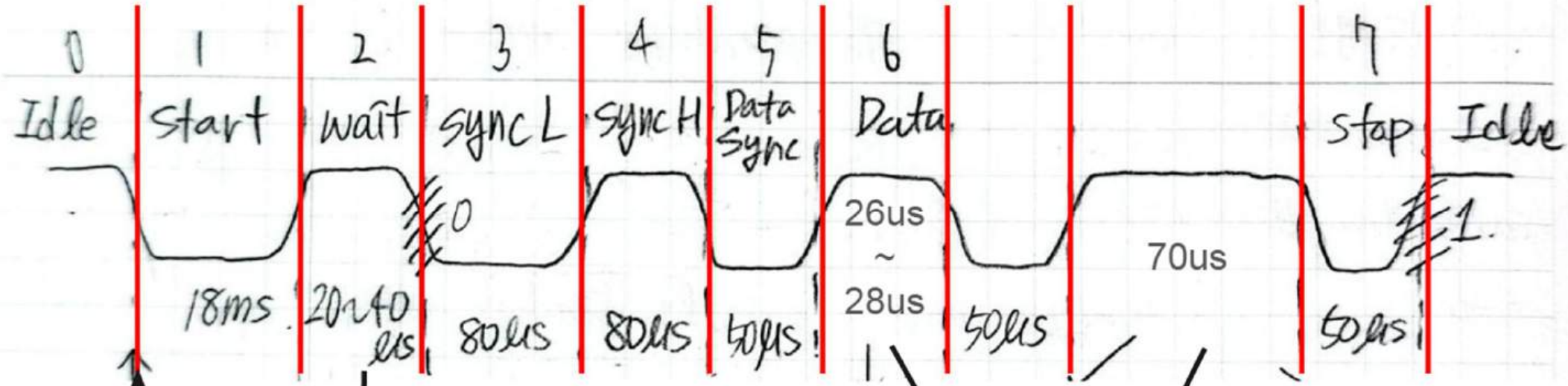


Figure 1 Typical Application

40bit = 습도 정수(8bit) + 습도 소수(8bit) + 온도 정수(8bit) + 온도 소수(8bit) + checksum(8bit)

통신 과정



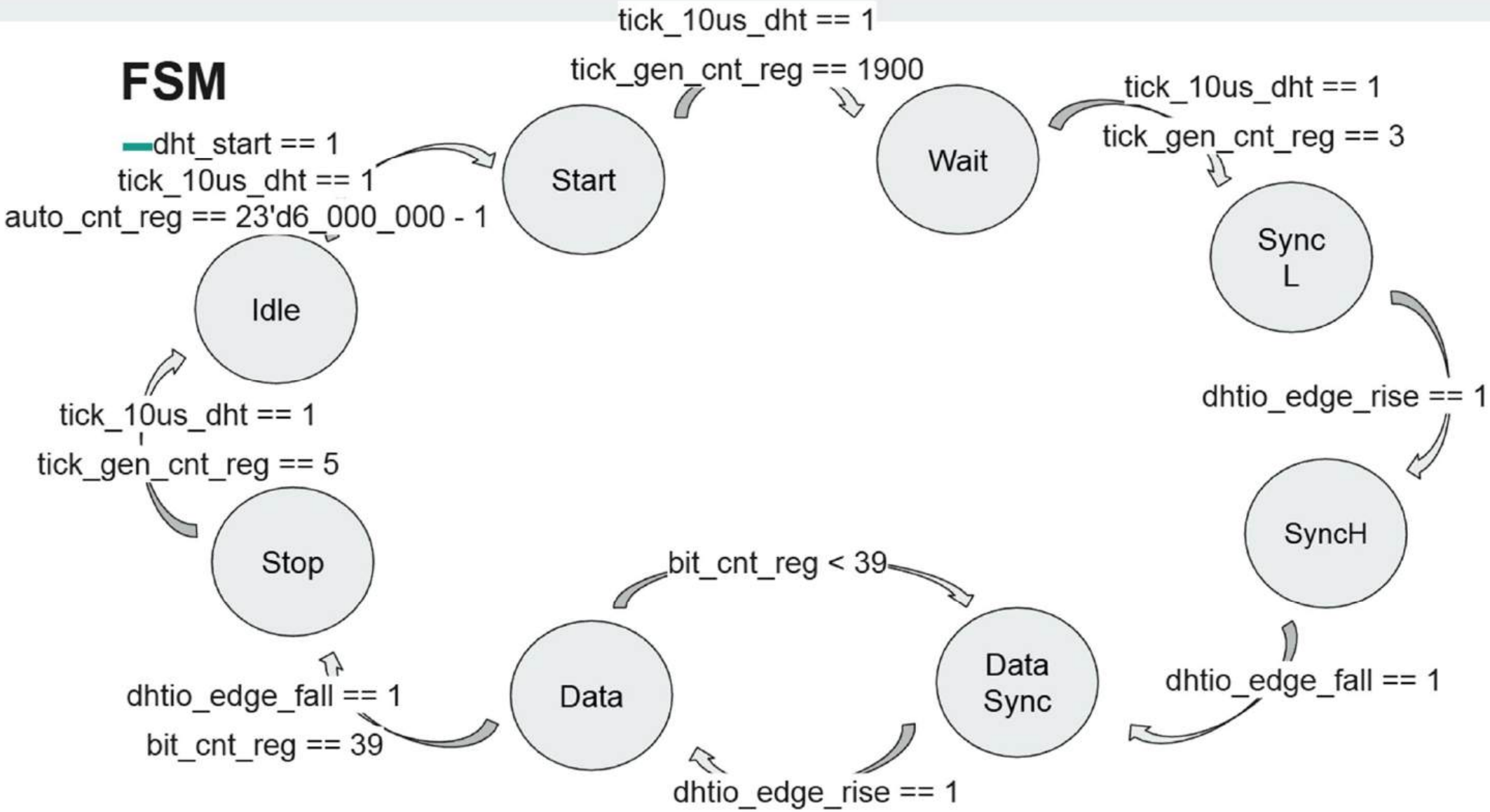


Start

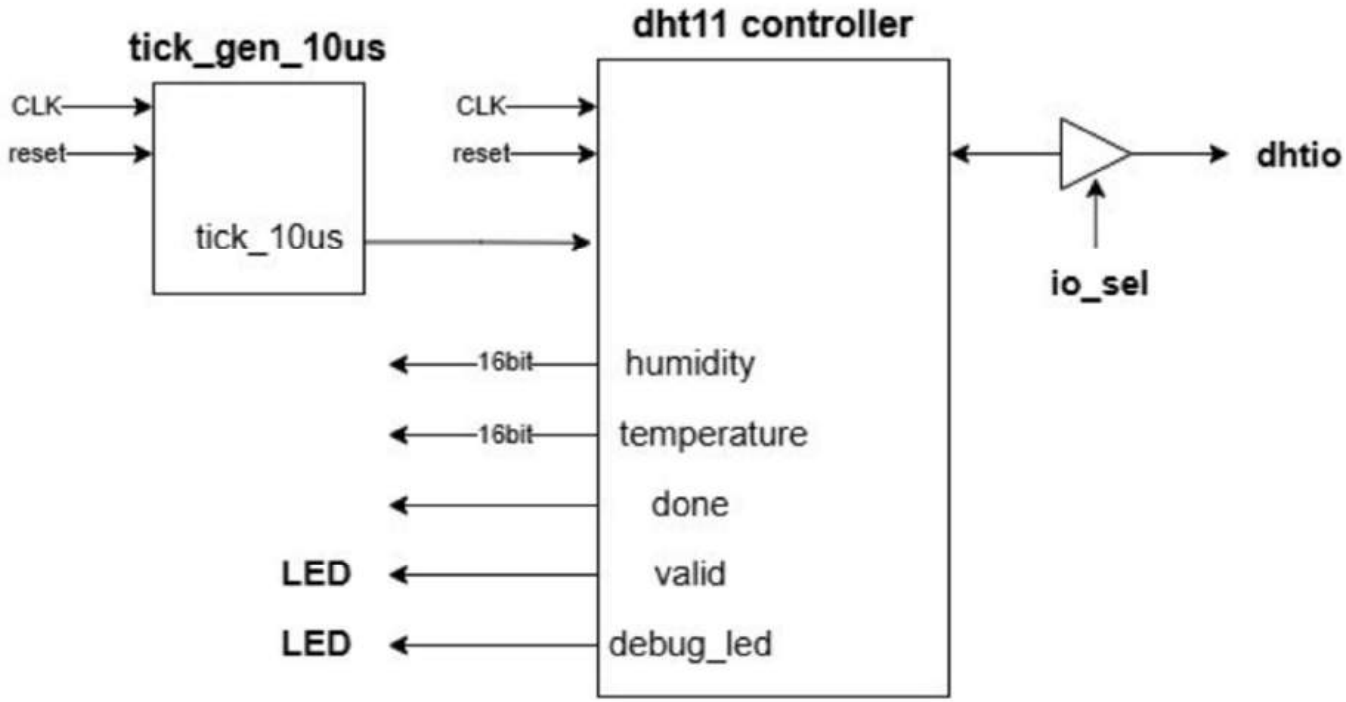
30us

50us 보다 작으면, **'0'**
50us 보다 크면, **'1'**
총 40bit

FSM

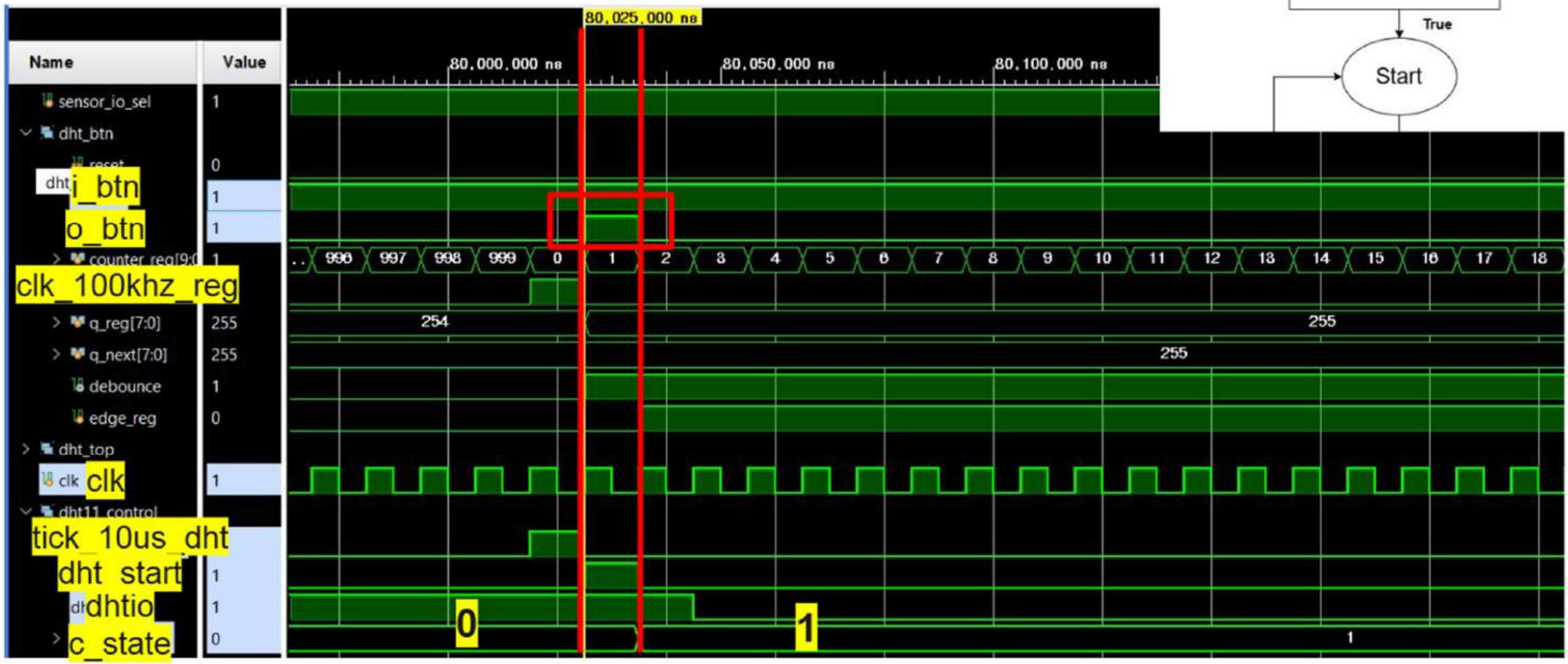

dht_start == 1
tick_10us_dht == 1
auto_cnt_reg == 23'd6_000_000 - 1
Idle
Start
tick_10us_dht == 1
tick_gen_cnt_reg == 1900
Wait
tick_10us_dht == 1
tick_gen_cnt_reg == 3
Sync L
dhtio_edge_rise == 1
SyncH
dhtio_edge_fall == 1
Data Sync
dhtio_edge_rise == 1
bit_cnt_reg < 39
Data
dhtio_edge_fall == 1
bit_cnt_reg == 39
Stop
tick_10us_dht == 1
tick_gen_cnt_reg == 5

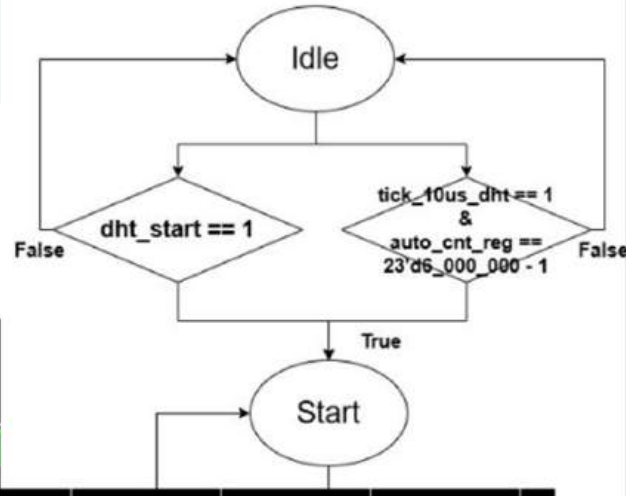
DHT 11_Block diagram




```
module dht11_top (
    input clk,
    input rst,
    input btn_r_start,

    output [15:0] humidity_data,
    output [15:0] temperature_data,
    output        dht_done,
    output        dht_valid,
    output [10:0] dht_debug_led,

    inout dhtio
);
```

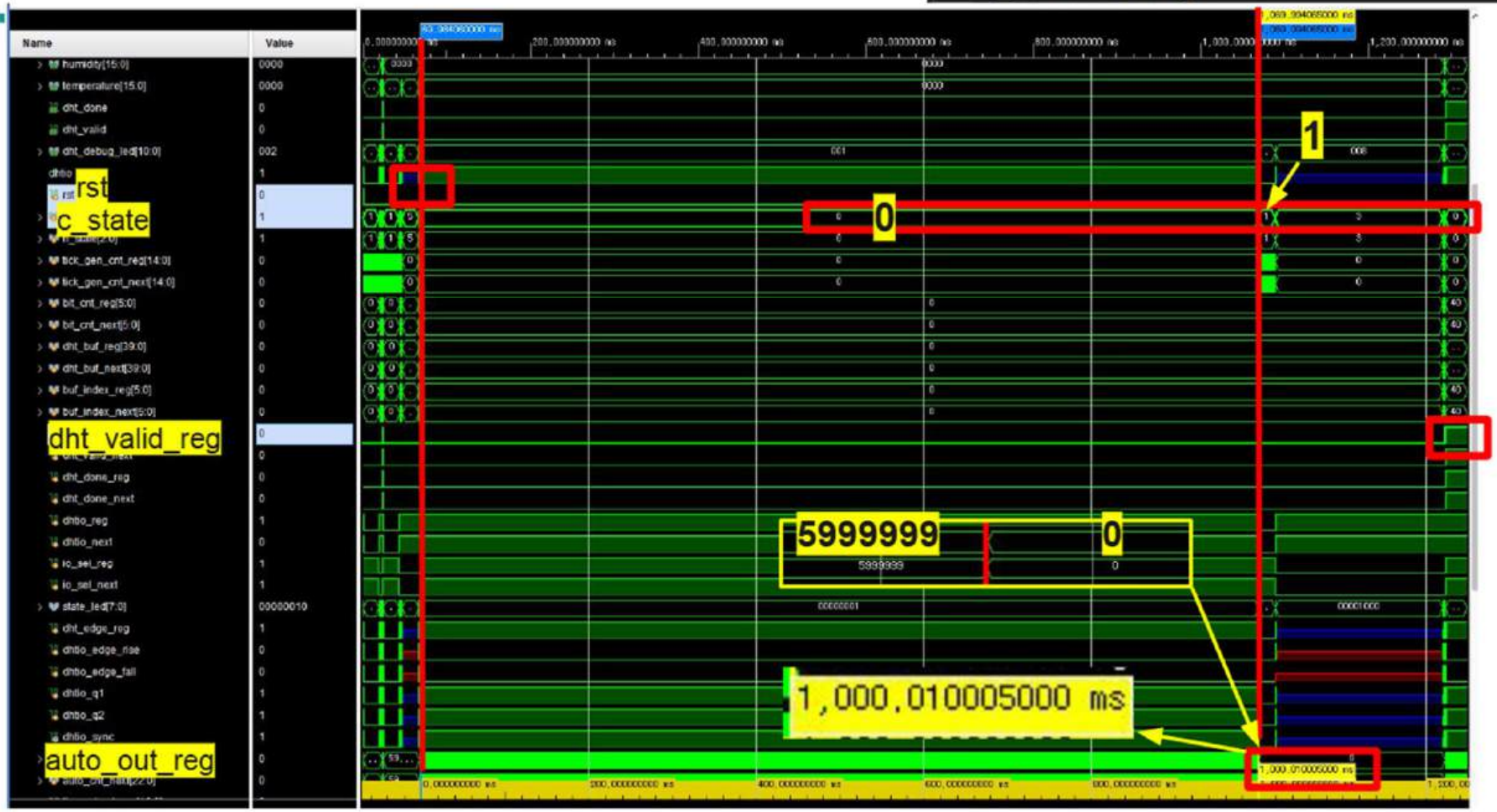
Button ‘r’ = DHT11 Start 신호



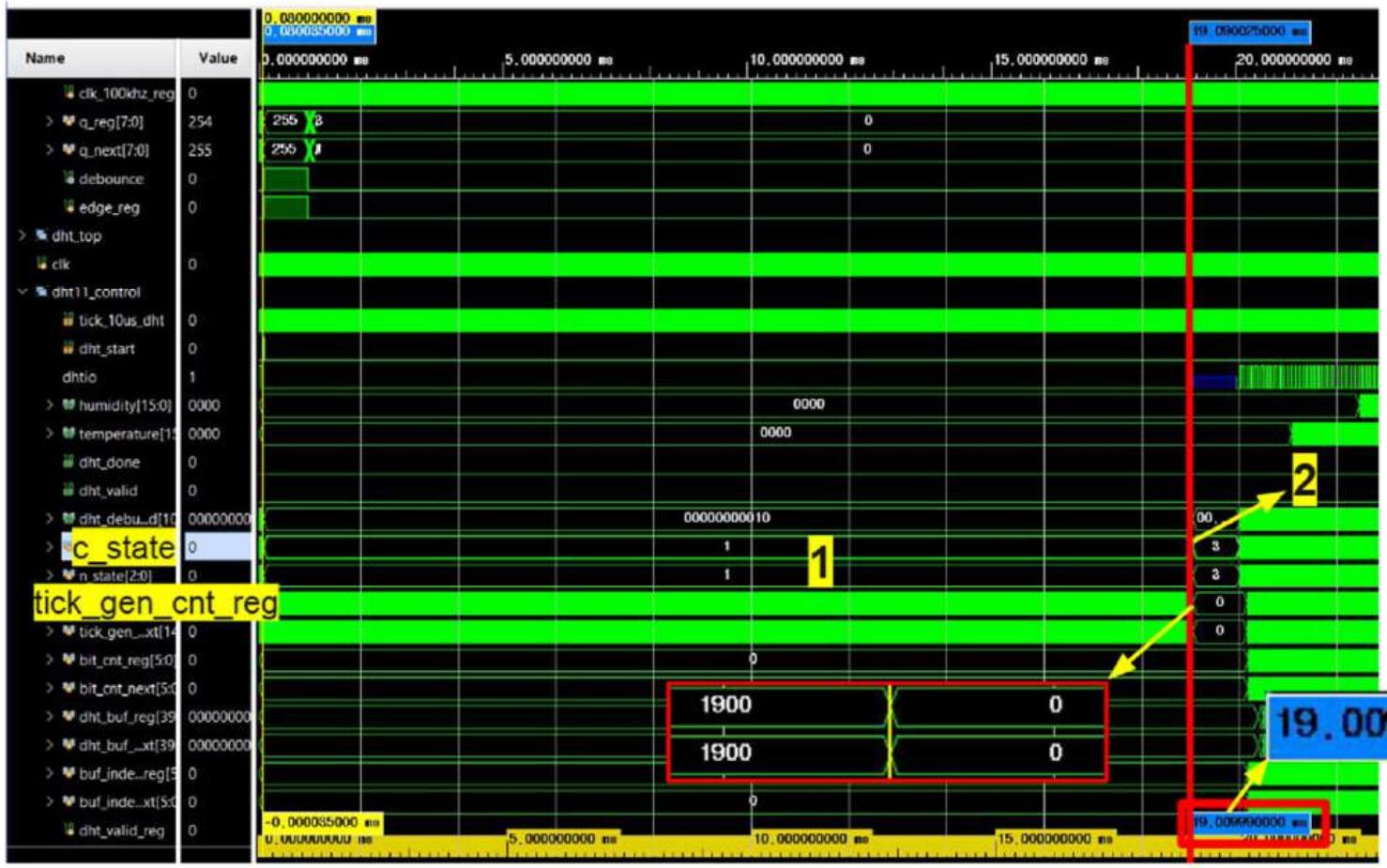


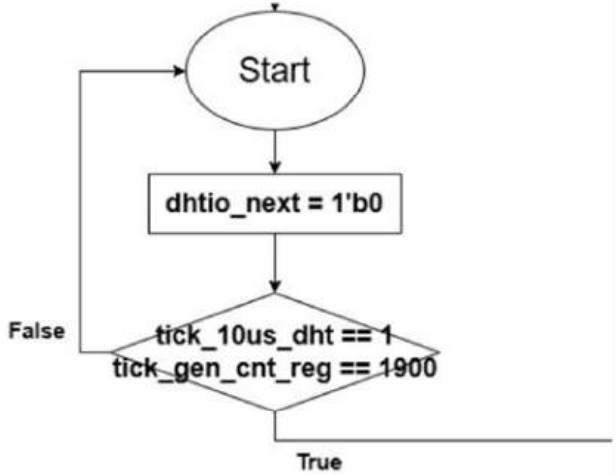
Reset 이후, 1초 뒤에 자동 START

```
auto_cnt_reg      <= 23'd6_000_000 - 100_000 - 1;
if (auto_cnt_reg == 23'd6_000_000 - 1) begin
    auto_cnt_next = 0;
    n_state = START;
```

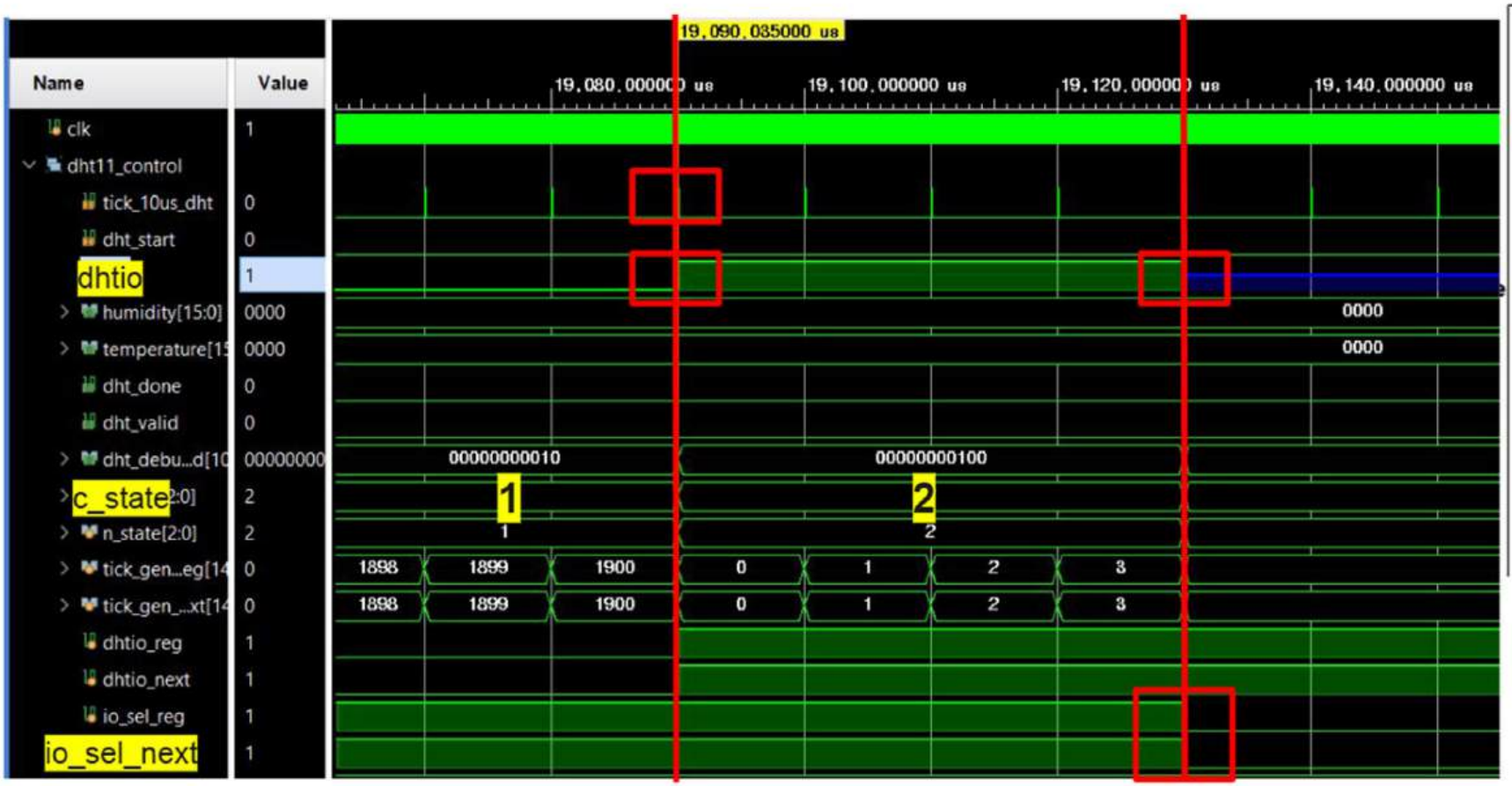


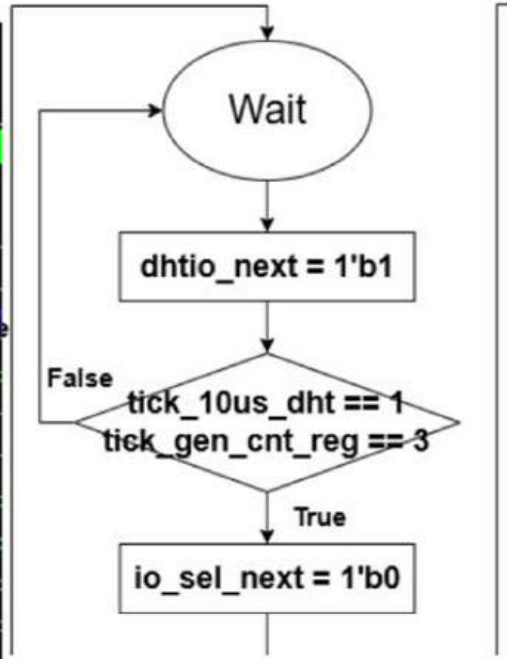
Start State → 19ms → Wait State



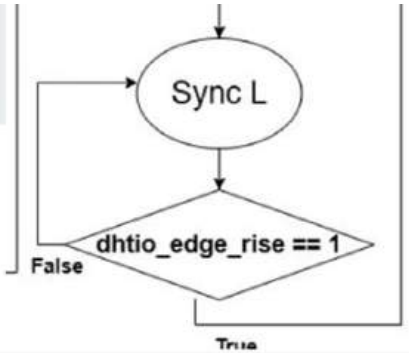


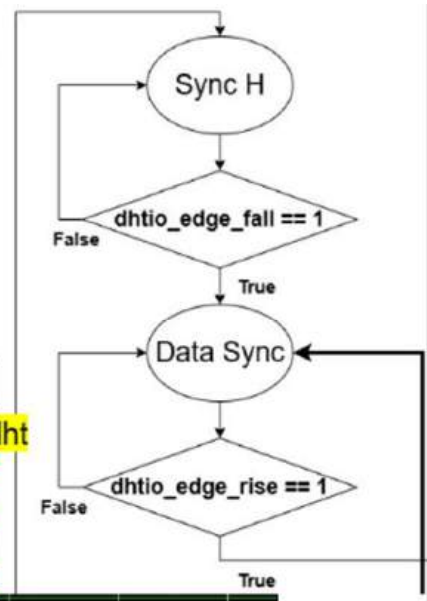
Wait State, inout인 dhtio 끊어짐

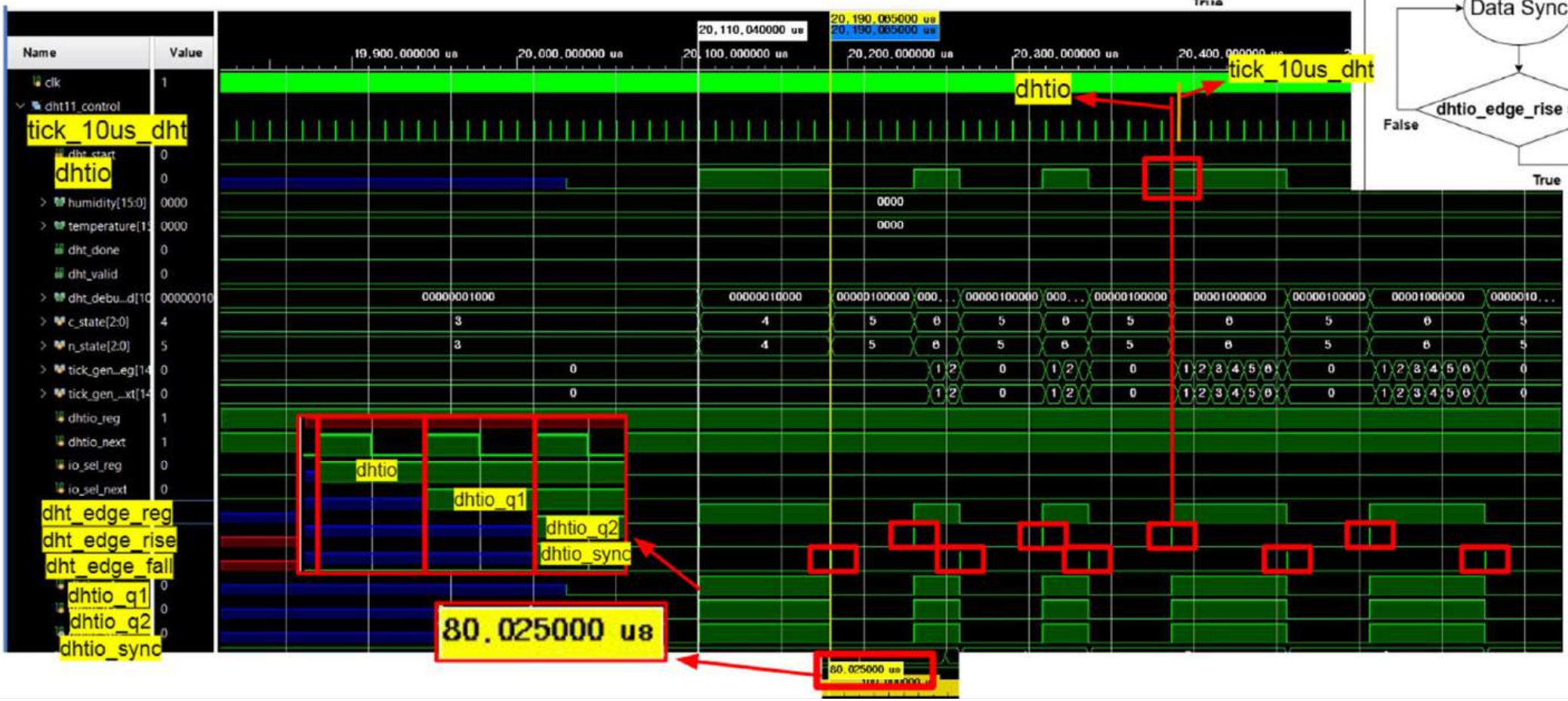




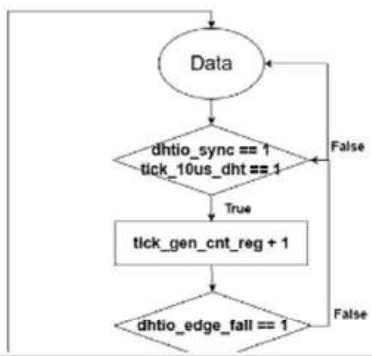
Synchronizer & Edge detection

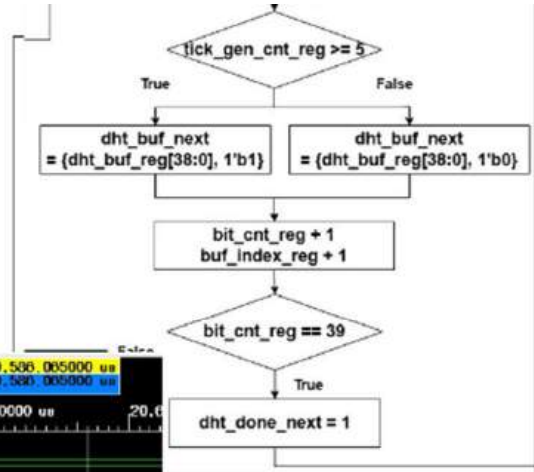


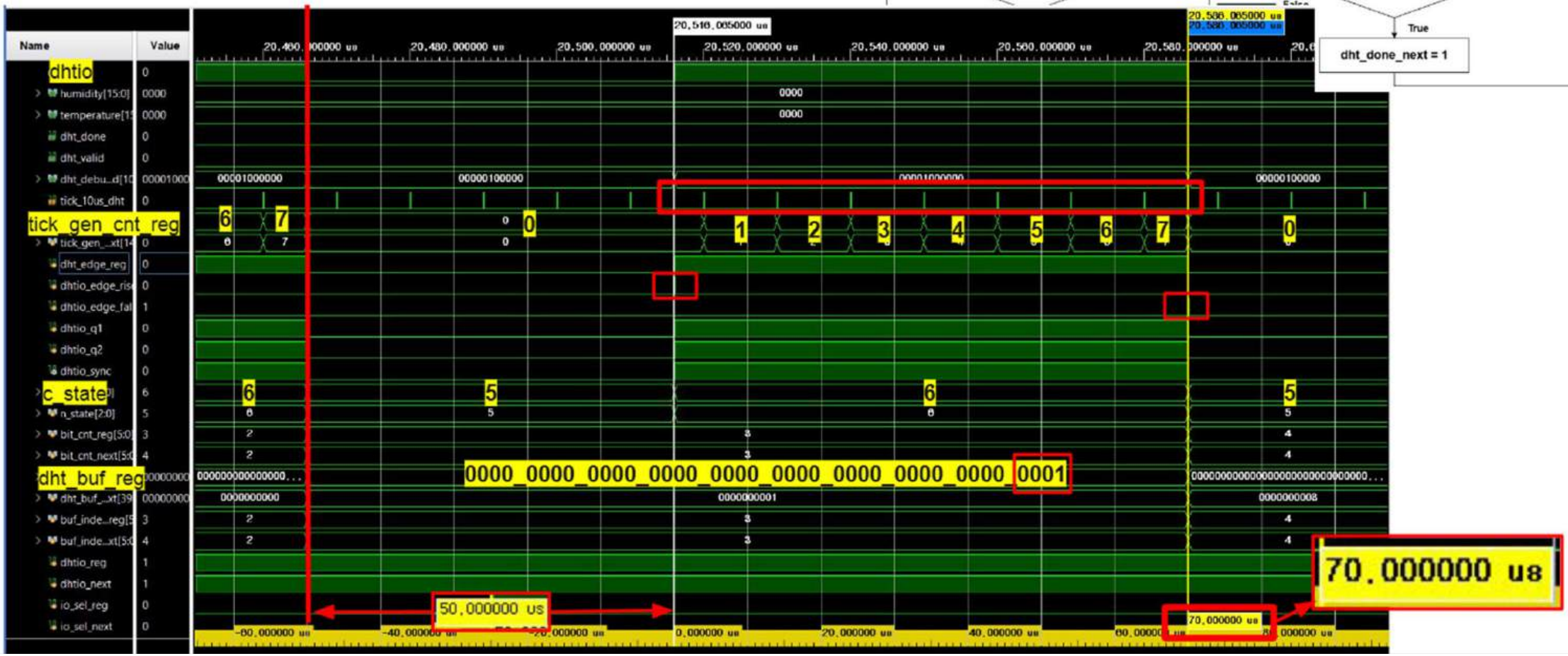




Data State, 0과 1 판별 → 1



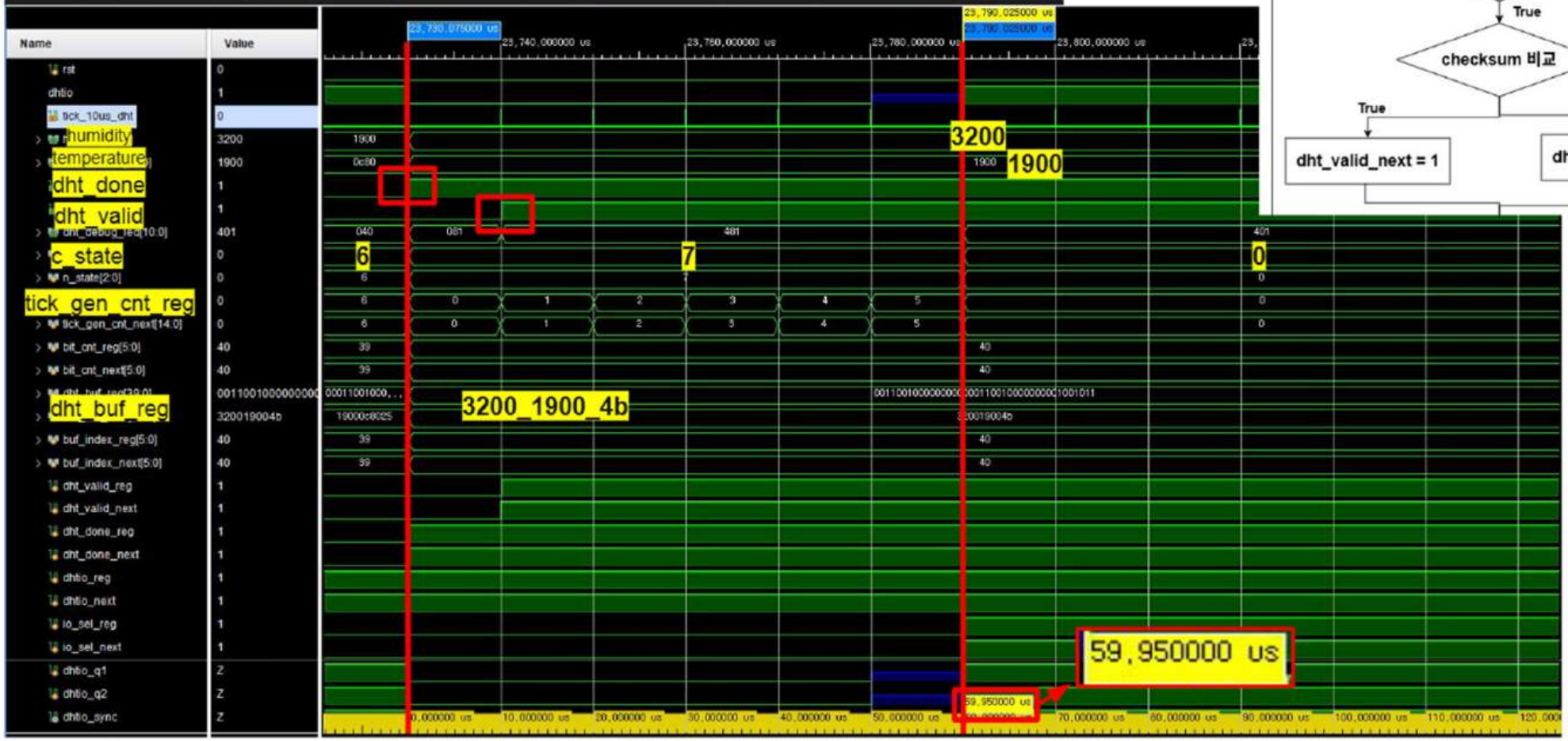


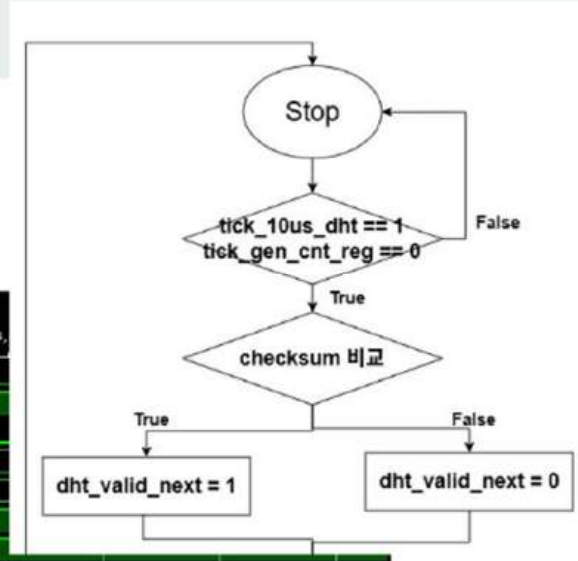


Data State, 0과 1 판별 → 0

Checksum Success → Valid = 1

```
dht11_sensor_data = {8'h32, 8'h00, 8'h19, 8'h00, 8'h4b};
```

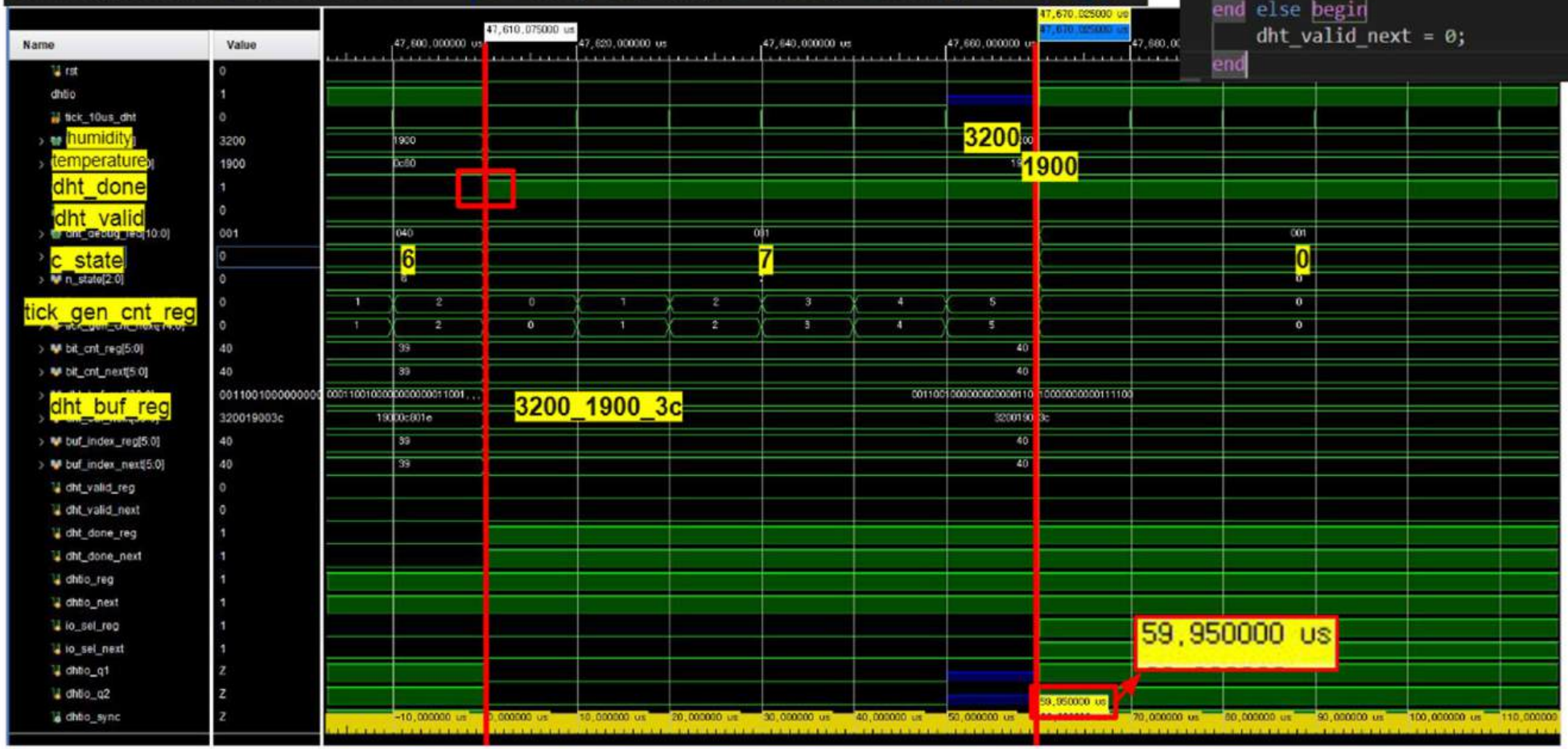




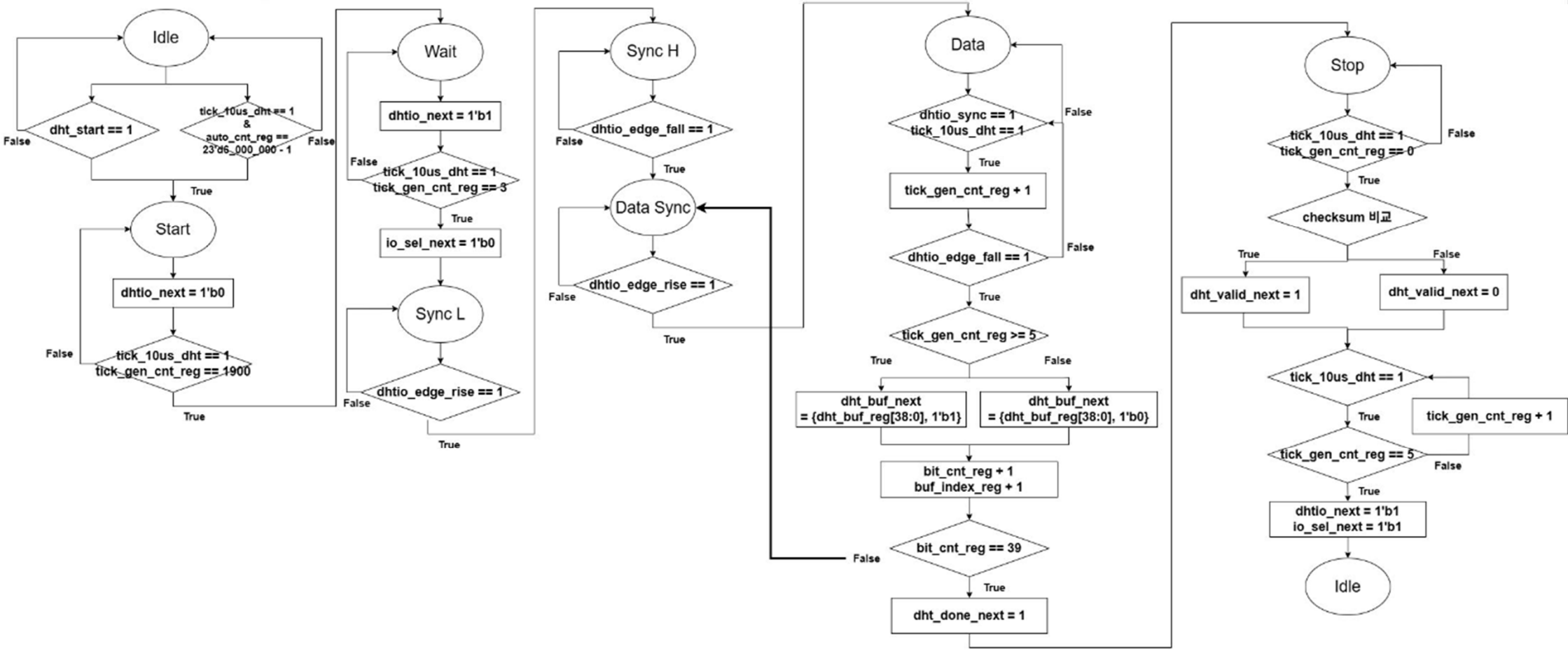
Checksum Fail → Valid = 0

```
dht11_sensor_data_fail = {8'h32, 8'h00, 8'h19, 8'h00, 8'h3c};
```

```
//checksum
if (dht_buf_reg[7:0] == (dht_buf_reg[39:32] +
                         dht_buf_reg[31:24] +
                         dht_buf_reg[23:16] +
                         dht_buf_reg[15:8])
                         & 8'hFF) begin
    dht_valid_next = 1;
end else begin
    dht_valid_next = 0;
end
```



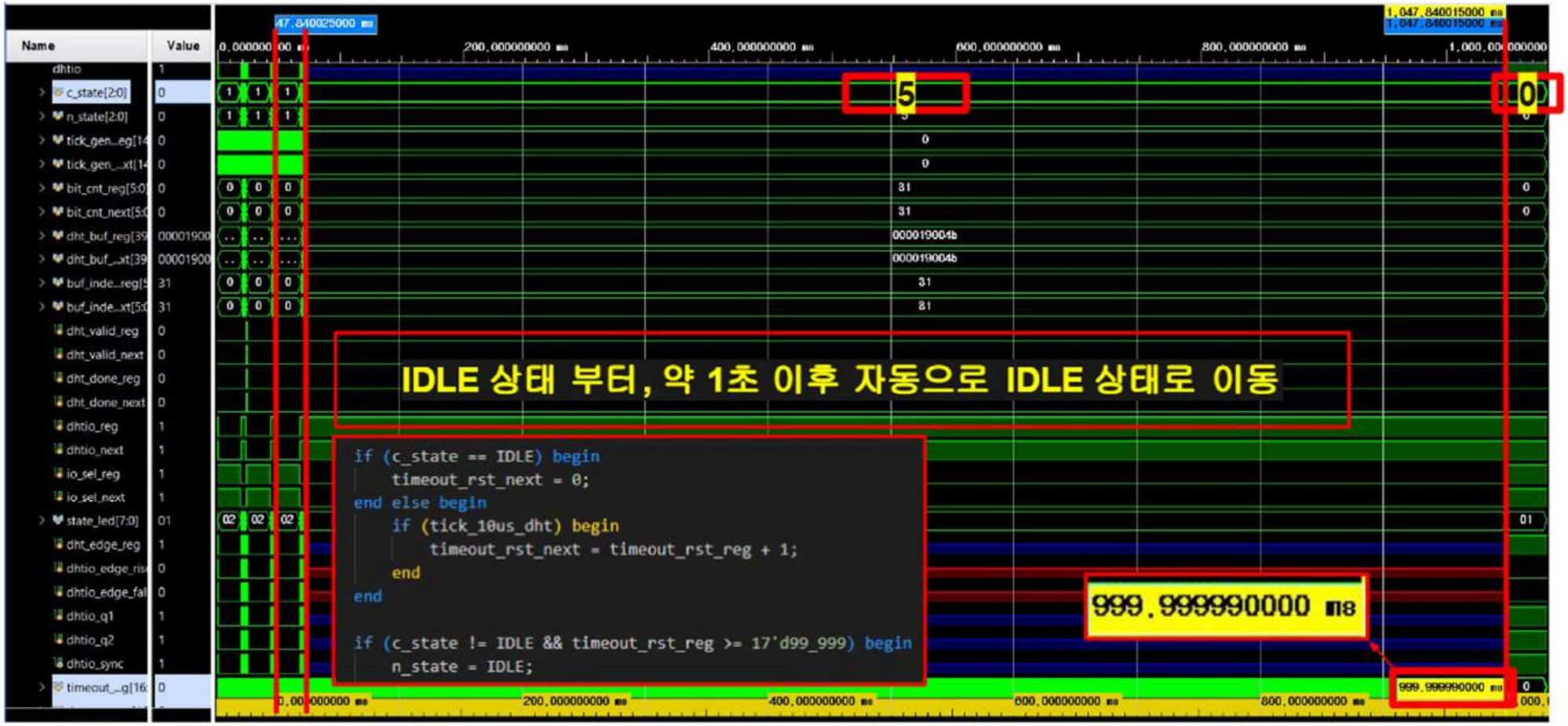
ASM


Idle
dht_start == 1
tick_10us_dht == 1
&
auto_cnt_reg ==
23'd6_000_000 - 1
False
True
Start
dhtio_next = 1'b0
tick_10us_dht == 1
tick_gen_cnt_reg == 1900
Wait
dhtio_next = 1'b1
tick_10us_dht == 1
tick_gen_cnt_reg == 3
io_sel_next = 1'b0
Sync L
dhtio_edge_rise == 1
Sync H
dhtio_edge_fall == 1
Data Sync
dhtio_edge_rise == 1
Data
dhtio_sync == 1
tick_10us_dht == 1
tick_gen_cnt_reg + 1
dhtio_edge_fall == 1
tick_gen_cnt_reg >= 5
dht_buf_next
= {dht_buf_reg[38:0], 1'b1}
dht_buf_next
= {dht_buf_reg[38:0], 1'b0}
bit_cnt_reg + 1
buf_index_reg + 1
bit_cnt_reg == 39
dht_done_next = 1
Stop
tick_10us_dht == 1
tick_gen_cnt_reg == 0
checksum 비교
dht_valid_next = 1
dht_valid_next = 0
tick_10us_dht == 1
tick_gen_cnt_reg + 1
tick_gen_cnt_reg == 5
dhtio_next = 1'b1
io_sel_next = 1'b1
Idle

State 멈춤 현상 발생 시, Time out Reset

bit 저장 31에서 멈춤으로 인해, Data Sync 상태에서 멈춤

State 멈춤 현상 발생 시, Time out Reset



참고문헌

Ultrasonic Ranging Module HC – SR04 (datasheet)
Tech support : sevices@elecfreaks.com

DHT11 Humidity & Temperature Sensor (datasheet)
Manufacturer: D-Robotics